

07.4: Big Data Architectures



UMD DATA605 - Big Data Systems

07.4: Big Data Architectures

Instructor: Dr. GP Saggese, gsaggese@umd.edu

1 / 13

2 / 13: Software Testing

Software Testing

- **Goal**
 - Evaluate functionality, reliability, performance, and security of a product to ensure it meets requirements
 - Software testing is critical in development
- **Adages**
 - *“If it’s not tested, it doesn’t work”*
 - *“Debugging is 2x harder than writing code”*
 - Corollary: *“If you do your best to write code, how can you debug it?”*
- **Many different types of testing**
 - What do you test?
 - From what point of view?
 - ...

2 / 13

- **Software Testing**
- **Goal**
 - The main aim of software testing is to check if a product works as expected. This involves looking at how well it functions, how reliable it is, how fast it runs, and how secure it is. By doing this, we can make sure the software meets the needs and expectations set out at the beginning of the project.
 - Testing is a crucial part of developing software. Without it, we can’t be sure that the software will work correctly when it’s used by real people.
- **Adages**
 - The saying *“If it’s not tested, it doesn’t work”* highlights the importance of testing. It suggests that until software is tested, we can’t be confident that it will function properly.
 - Another saying, *“Debugging is 2x harder than writing code”*, points out that finding and fixing problems in code is often more difficult than writing the code in the first place. The corollary, *“If you do your best to write code, how can you debug it?”*, humorously suggests that even if you write code carefully, debugging remains a challenging task.
- **Many different types of testing**
 - There are various aspects of software that can be tested. This includes checking different features, looking at the software from different perspectives, and more.
 - Questions like “What do you test?” and “From what point of view?” are important because they guide the testing process. They help determine which parts of the software need attention and how to approach testing them effectively.

3 / 13: What Are You Testing?

What Are You Testing?

- **Unit testing**
 - Test individual components for correct functionality in isolation
- **Integration testing**
 - Ensure components work together as expected
 - Detect interface defects
- **System testing**
 - Evaluate a fully integrated system's compliance with requirements

3 / 13

- **What Are You Testing?**
 - **Unit testing**
 - *Unit testing* is about checking the smallest parts of your code, like functions or methods, to make sure they work correctly on their own.
 - This type of testing is crucial because it helps catch bugs early in the development process, making it easier and cheaper to fix them.
 - By isolating each component, you can ensure that each part of your code does exactly what it's supposed to do without interference from other parts.
 - **Integration testing**
 - *Integration testing* focuses on making sure that different parts of your application work well together.
 - This is important because even if individual components work perfectly on their own, they might not interact correctly when combined.
 - It helps identify issues related to the interfaces and interactions between integrated units, ensuring that the system as a whole functions smoothly.
 - **System testing**
 - *System testing* is about evaluating the entire application to ensure it meets the specified requirements.
 - This type of testing is performed on a complete, integrated system to verify that it behaves as expected in real-world scenarios.
 - It is the final step before the application is delivered to the user, ensuring that all components work together seamlessly and the system fulfills its intended purpose.

4 / 13: How Are You Testing?

How Are You Testing?

- **Smoke/sanity testing**
 - Quick check of functionalities to ensure main functions work
 - E.g., decide if a new build is stable
 - E.g., application doesn't crash on launch
- **Regression testing**
 - Ensure new changes don't affect existing functionality
- **Acceptance testing**
 - Final testing phase before release
 - More common in waterfall than Agile
- **Performance testing**
 - Load, stress, and spike testing
- **Security testing**
 - Identify vulnerabilities, threats, and risks
- **Usability testing**
 - Assess ease of use for end-users
 - E.g., ui/UX
- **Compatibility testing**
 - Check compatibility with browsers, database versions, OS, mobile devices

4 / 13

- **Smoke/Sanity Testing**
 - This is a *quick check* to ensure that the main functions of an application are working correctly. It's like a preliminary test to see if the software is stable enough for further testing.
 - For example, after a new build of the software is created, smoke testing helps decide if it's stable enough to proceed with more detailed testing.
 - A simple check could be to see if the application launches without crashing, ensuring that the basic functionality is intact.
- **Regression Testing**
 - This type of testing is crucial to make sure that new changes or updates to the software do not negatively impact the existing features.
 - It involves re-running previous tests to confirm that the old functionalities still work as expected after the new changes.
- **Acceptance Testing**
 - This is the *final testing phase* before the software is released to the users. It ensures that the software meets the business requirements and is ready for deployment.
 - Acceptance testing is more commonly associated with the waterfall model of software development, where phases are completed sequentially, compared to Agile, which is more iterative.
- **Performance Testing**
 - This involves testing the software under various conditions to see how it performs. It includes load testing (how the software handles many users), stress testing (how it performs under extreme conditions), and spike testing (how it handles sudden increases in load).

- **Security Testing**
 - The goal here is to identify any vulnerabilities, threats, or risks in the software. This ensures that the software is secure from potential attacks and protects user data.
- **Usability Testing**
 - This type of testing assesses how easy and user-friendly the software is for end-users. It focuses on the user interface (UI) and user experience (UX) to ensure that users can navigate and use the software effectively.
- **Compatibility Testing**
 - This ensures that the software works well across different environments, such as various web browsers, database versions, operating systems, and mobile devices. It is important for ensuring a consistent user experience across different platforms.

5 / 13: CI / CD

CI / CD

- **Continuous integration (CI)**
 - Merge code changes into a central repository multiple times a day
 - Automate build and test after each change
 - Add code with unit tests
 - *Goal:* Detect and fix integration errors quickly
- **Continuous deployment (CD)**
 - Automatically deploy code changes to production
 - Without human intervention
 - After build and test phases pass
 - *Goal:* Deliver features, bug fixes, and updates continuously
- **Examples**
 - GitHub Actions, GitLab Workflows, AWS Code, Jenkins

5 / 13

- **Continuous Integration (CI)**
 - *Merge code changes into a central repository multiple times a day:* This means that developers frequently update the main codebase with their latest changes. By doing this often, it helps to catch any conflicts or issues early on.
 - *Automate build and test after each change:* Every time new code is added, the system automatically compiles the code and runs tests to ensure everything works as expected. This helps maintain code quality.
 - *Add code with unit tests:* Developers write small tests for their code to check if individual parts work correctly. These tests are run automatically to catch errors early.
 - *Goal:* The main aim is to quickly find and fix any problems that arise when new code is integrated, making the development process smoother and more efficient.
- **Continuous Deployment (CD)**

- *Automatically deploy code changes to production:* Once the code passes all tests, it is automatically released to users without needing someone to manually approve it.
 - *Without human intervention:* This reduces delays and human errors, ensuring that updates reach users faster.
 - *After build and test phases pass:* Only code that has been verified to work correctly is deployed, maintaining system stability.
- *Goal:* The objective is to continuously provide users with new features, improvements, and bug fixes, enhancing the overall user experience.
- **Examples**
 - *GitHub Actions, GitLab Workflows, AWS Code, Jenkins:* These are popular tools that help automate the CI/CD process. They provide the necessary infrastructure to manage code changes, run tests, and deploy updates efficiently.

6 / 13: RESTful API

RESTful API

- **REST API**
 - REST = REpresentational State Transfer
 - Style of building API for web services / distributed systems
- **Uniform interface**
 - Refer to resources
 - E.g., document, services, URI, persons
 - Use HTTP methods
 - E.g., **GET**, **POST**, **PUT**, **DELETE**
 - Naming convention, link format
 - Response (XML or JSON)
- **Stateless**
 - Each request contains all necessary information
 - No shared state
 - Inspired by HTTP (modulo cookies)

6 / 13

- **RESTful API**
 - **@REST API@**
 - *REST* stands for **REpresentational State Transfer**. It's a set of guidelines for creating web services that allow different systems to communicate over the internet.
 - REST is not a protocol but a style or architectural pattern used to design networked applications. It helps in building APIs that are scalable and easy to maintain.
- **Uniform interface**
 - RESTful APIs use a **uniform interface** to interact with resources. A resource can be anything like a document, a service, or even a person, and is identified by a URI (Uniform Resource Identifier).
 - HTTP methods are used to perform actions on these resources. For example, **GET** re-

- retrieves data, **POST** sends new data, **PUT** updates existing data, and **DELETE** removes data.
- Consistent naming conventions and link formats are important for clarity and ease of use.
- Responses from RESTful APIs are typically in XML or JSON format, which are both human-readable and machine-readable.
- **Stateless**
 - RESTful APIs are **stateless**, meaning each request from a client to a server must contain all the information needed to understand and process the request.
 - There is no stored context on the server between requests, which simplifies the server design and improves scalability.
 - This concept is inspired by HTTP, which is inherently stateless, although cookies can be used to maintain session information if needed.

7 / 13: RESTful API

RESTful API

- **Cacheable**
 - APIs label response data as cacheable or non-cacheable
 - Client can reuse cacheable responses
 - Increase scalability and performance
- **Layered system**
 - Each layer interfaces only with the immediate layer
 - E.g., in a tier application

WHAT IS A REST API?

```

graph LR
    CLIENT[CLIENT] -- "HTTP (GET, POST, DELETE, PUT)" --> URL[URL: /surveys, /surveys/123, /surveys/123/resp...]
    URL --> SERVER[SERVER]
    SERVER -- "JSON" --> CLIENT
    
```

7 / 13

- ****@Cacheable@****
 - RESTful APIs can specify whether their responses are *cacheable* or not. This means that once a client receives a response, it can store it temporarily and reuse it for future requests, reducing the need to repeatedly fetch the same data from the server.
 - By reusing cacheable responses, clients can improve efficiency, as they don't need to wait for the server to process the same request again.
 - This approach enhances the *scalability* and *performance* of applications by reducing server load and network traffic.
- **@Layered system@**
 - In a layered system architecture, each layer only interacts with the layer directly adjacent to it. This separation of concerns allows for more modular and maintainable code.

- For example, in a multi-tier application, the presentation layer might interact with the business logic layer, which in turn interacts with the data access layer. This structure helps in isolating issues and making changes without affecting the entire system.

8 / 13: Stages of Deployment

Stages of Deployment

- **Software is deployed through several environments**
 - Each environment tests, validates, and prepares software for release to end user
- **Development environment (Dev)**
 - Individual for each developer or feature team
 - *Goal:* Developers write and initially test code
- **Testing**
 - Aka “Quality Assurance (QA)”
 - Mirrors production environment to perform under similar conditions
 - *Goal:* Systematic testing to uncover defects and ensure quality
- **Pre-prod**
 - Aka staging
 - Final testing phase before deployment to production
 - Replica of production environment for final checks and stakeholder review
- **Production (Prod)**
 - Live environment where software is available to end users
 - Optimized for security, performance, and scalability
 - Focus on uptime, user experience, and data integrity

8 / 13

- **Stages of Deployment**
 - Software deployment involves moving software through different environments before it reaches the end user. Each environment serves a specific purpose to ensure the software is ready for release. This process helps in identifying and fixing issues early, ensuring a smooth and reliable software experience for users.
- **Development environment (Dev)**
 - Each developer or team has their own development environment. This is where they write and initially test their code. The *goal* here is to allow developers to experiment and make changes without affecting others. It’s a safe space for innovation and problem-solving.
- **Testing**
 - Also known as “Quality Assurance (QA),” this environment is designed to mimic the production environment as closely as possible. The *goal* is to conduct thorough testing to find and fix defects. This stage is crucial for maintaining software quality and ensuring that it performs well under conditions similar to those it will face in the real world.
- **Pre-prod**
 - Often referred to as staging, this is the last testing phase before the software goes live. It is a replica of the production environment and is used for final checks and reviews by stakeholders. This stage ensures that everything works as expected and that any last-minute issues are addressed.

- **Production (Prod)**
 - This is the live environment where the software is available to end users. It is optimized for security, performance, and scalability. The focus here is on maintaining uptime, providing a good user experience, and ensuring data integrity. This is the final stage where the software fulfills its intended purpose.

9 / 13: Semantic Versioning

Semantic Versioning

- **Semantic versioning** conveys meaning about changes to software
 - Systematic approach
 - Communicate to users potential impact of updating to a new version
- **Increment version**
 - Major Version **X.y.z**
 - Incompatible API changes
 - Significant updates that may break backward compatibility
 - Minor Version **x.Y.z**
 - Backward-compatible enhancements
 - Significant new features that don't break existing functionalities
 - Patch Version **x.y.Z**
 - Backward-compatible bug fixes that address incorrect behavior
 - Pre-release Version:
 - Denote a pre-release version that might not be stable
 - E.g., **1.0.0-alpha**, **1.0.0-beta**
 - Releases for testing and feedback, not for production use
 - Build Metadata
 - Optional metadata to denote build information or environment

9 / 13

- **Semantic Versioning**
 - *Semantic versioning* is a way to give meaning to version numbers in software. This helps users understand what changes have been made and what impact updating might have.
 - It provides a systematic approach to versioning, making it easier for developers and users to communicate about software updates.
 - By using semantic versioning, developers can clearly communicate the potential impact of updating to a new version, helping users make informed decisions.
- **Increment Version**
 - **Major Version X.y.z**
 - This number changes when there are incompatible API changes. These are significant updates that might break backward compatibility, meaning older versions of the software might not work with the new version.
 - **Minor Version x.Y.z**
 - This is updated for backward-compatible enhancements. It includes significant new features that do not disrupt existing functionalities, ensuring that the software continues to work as expected.
 - **Patch Version x.y.Z**
 - This is for backward-compatible bug fixes. These updates address incorrect behavior

without adding new features or breaking existing ones.

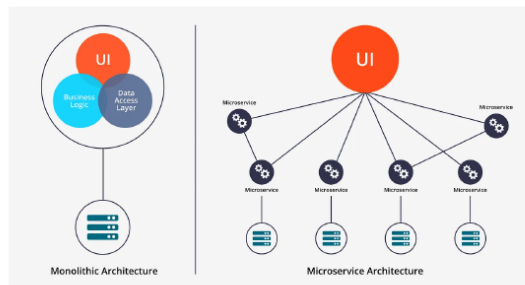
- **Pre-release Version**
 - These versions indicate a pre-release that might not be stable, such as 1.0.0-alpha or 1.0.0-beta. They are meant for testing and feedback, not for production use.
- **Build Metadata**
 - This is optional information that provides details about the build or environment, like 1.0.0+20210313120000 or 1.0.0+f8a34b3228c. It helps in identifying specific builds without affecting version precedence.

10 / 13: Microservices vs Monolithic Architecture

Microservices vs Monolithic Architecture

- **Many styles of building complex systems**

- *Monolith*: all features in one deployable unit
- *Microservices*: many small services collaborating over the network



- **Heuristics**

- Start from business domains, not technology layers
- Align service boundaries with independent business capabilities
- E.g., online shop with separate services for catalog, cart, payment, shipping

10 / 13

- **Many styles of building complex systems**
 - **Monolith**: In a monolithic architecture, all features and components of an application are bundled into a single deployable unit. This means the user interface, business logic, and data access layers are tightly integrated. While this can simplify deployment and development initially, it can become cumbersome as the application grows, making it harder to update or scale specific parts independently.
 - **Microservices**: This approach involves breaking down an application into smaller, independent services that communicate over a network. Each service is focused on a specific business capability, allowing for more flexibility, scalability, and easier maintenance. Changes can be made to individual services without affecting the entire system.
- **Heuristics**
 - **Start from business domains, not technology layers**: When designing microservices, it's crucial to focus on the business needs and capabilities rather than the technical structure. This ensures that each service aligns with a specific business function.
 - **Align service boundaries with independent business capabilities**: Each mi-

crosservice should represent a distinct business capability. For example, in an online shop, separate services could handle catalog management, shopping cart operations, payment processing, and shipping logistics. This alignment allows for more efficient scaling and development tailored to specific business needs.

11 / 13: Monolithic Architecture

Monolithic Architecture

- **Monolith** = all features in one deployable unit
- **Pros:**
 - *Simplicity*: Simpler to develop, test, deploy, and scale as a single unit
- **Cons**
 - *Tightly coupled components*: Components run in the same process, leading to scalability and resilience issues
 - *Technology stack uniformity*: Developed with a single technology stack, limiting flexibility
 - *Deployment complexity*: Updates require redeploying the entire application
 - *Single point of failure*: Issues in any module can affect the entire application

11 / 13

- **Monolithic Architecture**
 - *Monolith* means that all the features of an application are bundled together into one single deployable unit. This is like having all your eggs in one basket, where everything is interconnected and runs as a single entity.
- **Pros:**
 - *Simplicity*: One of the main advantages of a monolithic architecture is its simplicity. Since everything is in one place, it's easier to develop because you don't have to worry about integrating different parts. Testing is straightforward because you can test the whole application at once. Deployment is also simpler because you only have to deploy one unit, and scaling can be done by replicating the entire application.
- **Cons:**
 - *Tightly coupled components*: In a monolithic architecture, all components are interconnected and run in the same process. This can lead to problems with scalability because you can't scale individual parts independently. It also affects resilience because if one part fails, it can bring down the whole system.
 - *Technology stack uniformity*: You're limited to using a single technology stack for the entire application. This means you can't choose the best technology for each part of your application, which can limit flexibility and innovation.
 - *Deployment complexity*: Even if you make a small change to one part of the application,

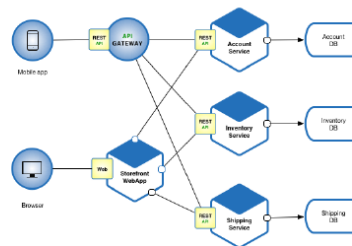
you have to redeploy the entire application. This can be time-consuming and risky, especially if the application is large.

- *Single point of failure*: If there's a problem in any module, it can affect the entire application. This means that a small bug can have a big impact, making the system less reliable.

12 / 13: Microservice Architecture

Microservice Architecture

- **Microservices** = many small services collaborating over the network
- **Cons**
 - Complex deployment
 - Requires tooling



- **Pros**:
 - *Modularity*: Small, independently deployable services with specific business functionality
 - *Scalability*: Scale services independently for efficient resource use based on demand
 - *Technology diversity*: Use the best technology stack for each service's functionality
 - *Deployment flexibility*: Supports continuous delivery and deployment for faster updates
 - *Resilience*: Isolate and address faults; one service failure doesn't affect the entire system

12 / 13

- **Microservices**: This refers to an architectural style where a system is divided into many small, independent services that communicate over a network. Each service is focused on a specific business function, allowing for more manageable and scalable systems.
- **Cons**:
 - **Complex deployment**: Managing numerous services can be challenging, as each service needs to be deployed and maintained separately.
 - **Requires tooling**: Effective management of microservices often requires specialized tools for deployment, monitoring, and orchestration.
- **Pros**:
 - **Modularity**: Each service is a small, independently deployable unit, which simplifies development and maintenance.
 - **Scalability**: Services can be scaled independently, allowing for efficient resource use based on demand.
 - **Technology diversity**: Different services can use different technology stacks, enabling the use of the best tools for each specific task.
 - **Deployment flexibility**: Supports continuous delivery and deployment, allowing for

faster updates and improvements.

- **Resilience:** Faults are isolated to individual services, so a failure in one service doesn't bring down the entire system.

The diagram illustrates how different services, such as Account, Inventory, and Shipping, interact through an API Gateway, demonstrating the modular and interconnected nature of microservices.

13 / 13: Microservices vs Monolith: Hype

Microservices vs Monolith: Hype

- Neither approach is a slam dunk!
 - You need to find the right granularity for your use case



13 / 13

- **Neither approach is a slam dunk!**
 - When deciding between *microservices* and *monolithic* architectures, it's important to understand that neither is a one-size-fits-all solution. Each has its own advantages and challenges.
 - The key is to determine the right level of granularity for your specific use case. This means understanding the needs of your application and how best to structure it to meet those needs efficiently.
- **Search Trends**
 - The graph shows the search interest over time for "Microservice vs Monolithic." It highlights how interest in these architectural styles has evolved, with microservices gaining more attention over the years.
 - This trend reflects the growing popularity of microservices, often due to their flexibility and scalability. However, it also suggests that monolithic architectures still hold relevance, especially for simpler or smaller applications.
 - The decision should be based on practical considerations rather than trends, focusing on what best suits the project's requirements and constraints.